

Session 3

Technical

Presentation Skills



A parent understands it; an engineer respects it.

Part	Topic	Time
01	Explaining technical ideas to non-technical people	8 min
02	Presenting your architecture: system, data flow, code	10 min
03	Being honest about AI, and handling questions	4 min
04	Mock presentation: your product architecture	8 min

Six words you will hear today.

Word	Plain meaning
Jargon	Words only specialists understand, like “hysteresis” or “ADC”.
Architecture	How a system is organised: its parts and how they connect.
System design	The big-picture plan: which parts, doing what, talking to whom.
Data flow	The path information takes — reading, decision, action.
Code map	A short table of the main parts of your code and what each does.
Level of detail	How far you zoom in: what it does, how parts connect, or the code.

Explain like a map app: zoom in only when needed.

Zoomed out you see the city; zoom in for streets; further for buildings.

Start with what it does. Zoom into the code only for a technical audience, or when someone asks.

Three levels of detail.

Level	What you show	Who needs it
1. What it does	“It waters your plant only when the soil is dry.”	Everyone
2. How the parts connect	Probe → board → pump: the block diagram	Curious listeners, teachers
3. How the code decides	Two thresholds, a bool, a few lines of code	Engineers, judges, interviewers

Swap jargon for pictures people know.

Jargon	Plain words or an analogy
Soil moisture sensor	“A probe that feels how wet the soil is — like your finger, but it never forgets.”
Threshold	“The line where it changes its mind: below 30 % wet, it waters.”
Hysteresis	“Like running a bath: on when the water’s low, off only when it’s high enough.”
State machine	“It’s always in one mood — watering or waiting — with clear rules for switching.”
–1 fail-safe	“If a sensor is unplugged, it plays safe and does nothing.”

Try it (3 min): explain your hardest idea to a partner in two sentences, with no jargon. They repeat it back. If they can’t, try another analogy.

Show your design in three views, far to near.

View 1: the system — your block diagram. View 2: the data flow — one reading's journey.

View 3: the code — a map, then the few lines that decide.

Never put a whole program on a slide.

Follow one reading like a parcel.

Step	Smart Garden
Sense	Every 2 seconds, the probe gives a raw reading.
Clean	A -1 becomes “act as if wet”; any other reading becomes % wet using your calibration.
Decide	Two thresholds and the current state choose: on, off, or keep going.
Act	The pump (today: the PUMP ON / PUMP OFF stand-in), and the Serial Monitor shows it.

Map the code before you show any of it.

Part of the code	What it does
DRY_VALUE, WET_VALUE	Your calibration readings
wetPercent()	Turns a raw reading into % wet
START, STOP	The two thresholds
watering	Remembers the state: on or off
loop()	Reads, decides and acts, every 2 seconds

Narrate the key lines in plain words.

```
1  if (!watering && wet < START) { watering = true;  Serial.println("PUMP ON"); }  
2  if (watering && wet > STOP)    { watering = false; Serial.println("PUMP OFF"); }
```

LINE 1 · STEP 1 OF 5

The pump is off and the soil is 25 % wet.

VARIABLES

```
wet = 25  
watering = false
```

SERIAL MONITOR



Narrate the key lines in plain words.

```
1  if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
2  if (watering && wet > STOP)    { watering = false; Serial.println("PUMP OFF"); }
```

LINE 1 · STEP 2 OF 5

Say: “If the pump is off and the soil is drier than 30 %, switch it on.” It is — so it switches on.

VARIABLES

wet = 25
watering = `true`

SERIAL MONITOR

PUMP ON

Narrate the key lines in plain words.

```
1  if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
2  if (watering && wet > STOP)    { watering = false; Serial.println("PUMP OFF"); }
```

LINE 2 · STEP 3 OF 5

Say: "If it's on and wetter than 60 %, switch it off." Still 25 %, so it keeps watering.

VARIABLES

wet = 25
watering = true

SERIAL MONITOR

PUMP ON

Narrate the key lines in plain words.

```
1  if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
2  if (watering && wet > STOP)    { watering = false; Serial.println("PUMP OFF"); }
```

LINE 1 · STEP 4 OF 5

Later the soil is 65 % wet. “The pump is already on, so this line does nothing...”

VARIABLES

wet = 65
watering = true

SERIAL MONITOR

PUMP ON

Narrate the key lines in plain words.

```
1  if (!watering && wet < START) { watering = true; Serial.println("PUMP ON"); }  
2  if (watering && wet > STOP)    { watering = false; Serial.println("PUMP OFF"); }
```

LINE 2 · STEP 5 OF 5

“...but it’s wetter than 60 %, so the pump switches off.”

VARIABLES

wet = 65
watering = *false*

SERIAL MONITOR

PUMP ON
PUMP OFF

Say what the AI did — and what you did.

Say	Example
What the AI did	“The Workspace AI drafted my first <code>wetPercent()</code> and suggested a default: <code>case.</code> ”
What you checked	“I compared every <code>incipe.*</code> call with the Wiki, compiled it, and tested it with dry soil and water.”
What you changed	“Its first draft treated <code>-1</code> as a real reading. I added the fail-safe.”

Four situations, four moves.

Situation	What to do
You understand it	Repeat it briefly so everyone hears it, then answer at the right level.
You don't understand it	"Could you say a bit more about what you mean?"
You can't answer it	"I don't know yet — here's how I'd find out..." Never invent an answer.
It's about a weakness	Admit it, then say what you'd do: "It can't water two pots yet; a second pump would fix that."

Your product architecture in six slides, about 5 minutes.

Slide	Show	Level
1. What it does	One sentence and a photo of your build	1
2. System	Your block diagram: sensors, board, actuators	2
3. Data flow	Sense → clean → decide → act, or your flowchart	2
4. Code	Your code map and the 2–5 lines that decide	3
5. AI usage	What it did, what you checked, what you changed	—
6. Next	One weakness and how you'd fix it	—

The audience plays three roles.

A parent asks “what does it do for me?”. An engineer asks “why two thresholds?”. An interviewer asks “what did the AI do, and what did you do?”

After each talk, each role asks one question. Keep firmware to your own code and the incipe.* calls from the Wiki.

Explain “hysteresis” to a grandparent in one sentence.

READ THE CODE

```
hysteresis = ?
```

ANSWER

A bath

WHY

“Tap on when the water’s low, off only when it’s high enough
— so it doesn’t keep flicking on and off.”

Non-technical audience: what do you show first — code or what it does?

READ THE CODE

A: the code

B: what it does

ANSWER

B

WHY

What it does is level 1. Code is level 3 — for a technical audience or when asked.

Your slide shows 60 lines of code. How do you fix it?

READ THE CODE

[60 lines of code]

ANSWER

Code map

WHY

A code map plus only the 2–5 lines that decide, large, narrated line by line.

Checkpoint 4

A judge asks something you don't know. What do you say?

READ THE CODE

```
Judge: "What happens  
at 0 °C?"
```

ANSWER

Don't know yet

WHY

"I don't know yet — here's how I'd find out." Never invent an answer.

What goes on your “AI usage” slide?

READ THE CODE

AI usage:

?
?
?

ANSWER

Did · checked · changed

WHY

What the AI did, what you checked (Wiki names, compiling, testing), and what you changed.

Three things to take away.

1

Start zoomed out — what it does — and zoom in only as far as your audience needs; swap jargon for analogies.

2

Present architecture in three views: the system, the data flow, and a code map with the few lines that decide.

3

Be honest about AI — what it did, what you checked, what you changed — and say “I don’t know yet” rather than invent.

Next — Session 4: Final pitch & career preparation. Polishing the deck and docs, a demo video for your portfolio, and the final showcase.